Molecular Dynamics Simulations of Lipid Membranes

Instructions for the
Biophysics Practical Course at
Goethe University Frankfurt

Version 2.4

Max Planck Institute of Biophysics

Ramachandra M. Bhaskara
Roberto Covino
Michael Gecht
Jürgen Köfinger
Martin Vögele
Marc Siggel
Balázs Fábián
Ainara Claveras

Frankfurt am Main, December 20, 2023

"In this house, we obey the laws of thermodynamics!"

(Homer Simpson)

Contents

1 Introduction

Nature employs membranes to partition and compartmentalize living systems. Biological membranes act as physical barriers to limit diffusion into and out of the compartment and regulate entry and exit of different types of molecules across membranes. Proteins embedded in membranes perform these essential gate keeping functions and sense chemicals, voltage, pH, mechanical stress, and other stimuli. Furthermore, the presence of proteins associated with energy conversion in almost all organisms is tightly associated with membranes, indicating that these systems provided platforms for multiple chemical reactions during the origin of life.

Phospholipids are the primary components of biological membranes. The head groups can be chemically linked to various small molecules such as choline, ethanolamine and others. Tail groups differ in length and the degree of saturation. This repertoire of combinations of different head and tail groups generates enormous diversity in the chemical nature of the lipids and their specific interactions with other cellular components like proteins. At the same time these chemically distinct lipids share the ability to form aggregates and higher order structures. The ability of phospholipids to aggregate and form micelles, vesicles, and bilayer structures is driven by the hydrophobic effect. The organization of individual lipid molecules within these aggregated structures such as a fluid membrane is generally dictated by their intermolecular interactions and interactions with the solvent.

The lipid bilayers can also be bent to form specific organelle shapes essential for functional compartmentalization. The structures adopted by membranes are stabilized by the molecular structure of the lipids and their interactions with proteins which are either peripheral or integral parts of the membrane itself.

Depending on its lipid composition, a given membrane may prefer to remain flat, or it may spontaneously adopt a curved shape. It requires energy, i.e., force, to deviate from these preferred shapes. It turns out that a membrane just four or five nanometers thick has sufficient rigidity to resist thermal fluctuations, for example, which is important for the maintenance of specific shapes of cell membranes. On the other hand, this rigidity is small enough to enable biologically feasible forces, on the order of piconewtons, to generate curvature of the membranes.

Curvature in bilayers is generated by an asymmetry of the two monolayers. If the two lipid monolayers that constitute the membrane have a similar structure then the lipid bilayer will remain flat. Changing the properties, molecular structure of lipids or structure of one monolayer will inevitably cause the membrane to bend. This can be achieved, for example, by binding

proteins to one membrane surface but not the other. Even low-affinity binding between a protein and a membrane monolayer can cause some curvature.

Molecular dynamics simulations allow us to study the formation of vesicles and bilayers, their structure and dynamics, and the dependence on lipid composition, for example. Many scientific questions relating to lipid membranes demand simulations on time and length scales that are inaccessible for fully atomistic simulations. Therefore, computationally efficient low resolution models of lipids, i.e., coarse-grained models like MARTINI, have been developed to study these system on current high performance computer clusters.

In this course, we will use molecular dynamics simulations to study the self-assembly of a lipid bilayer from a mixture of water and lipids. The files necessary to setup and run the simulation will be provided. A basic knowledge of Unix systems (Linux), and shell commands will be needed (see appendix). Help and support will be provided during the course. After the course, you should write a report. You can find our expectations and hints on writing a good report in the appendix.

This course is based on the tutorial[1] of Prof. Dr. Siewert-Jan Marrink at the Groningen Biomolecular Sciences and Biotechnology Institute and Zernike Institute for Advanced Materials at the University of Groningen [4]. The basics sections are based on a chapter in the master's thesis of one of the authors [7].

[1] http://cgmartini.nl/index.php/tutorials-general-introduction/bilayers

2 Molecular Dynamics Simulations

2.1 Introduction

In molecular dynamics simulations [1, 3], the system to be simulated is considered to contain N classical particles. These particles interact by mostly semi-empirical potentials. Pairwise potentials $V_{ij} = V(\vec{r}_{ij})$, for example, depend on the distance vectors between particles i and j at positions $\vec{r}_i$ and $\vec{r}_j$, respectively. By differentiating these potentials, one obtains the forces acting on the particles:

$$\vec{f}_i = \sum_{i \neq j} \vec{f}_{ij} = -\sum_{i \neq j} \frac{\partial V(\vec{r}_{ij})}{\partial \vec{r}_i} \,. \tag{1}$$

Because of this relation, we refer to interaction potential also as force fields. Their exact form shall be discussed in section 3. These forces enter Newton's equations for each particle i, i.e.,

$$m_i \ddot{\vec{r}}_i = \vec{f}_i \,. \tag{2}$$

The result are trajectories for all particles, i.e., we know positions and velocities of all particles at all times, which can be used for calculating macroscopic properties (observables) that can be compared to experiments. For example, we can calculate a time average of an on observable $A(\{\vec{r}_i, \dot{\vec{r}}_i\})$ depending on positions and velocities of particles via

$$\langle A \rangle = \frac{1}{T} \int_0^T \mathrm{d}t A(\{\vec{r}_i(t), \dot{\vec{r}}_i(t)\}) \,. \tag{3}$$

Moreover, in simulations we obtain insight into the structure and dynamics of systems at an atomistic level, which is is hard to achieve in experiments.

2.2 Integrating the Equations of Motion

In general, Newton's equations of motions cannot be solved analytically for many-body systems and we have to solve them numerically instead [3]. The most commonly used algorithms are integrators of Runge-Kutta type as well as the Verlet algorithm and its derivates. We introduce the latter category of algorithms here.

The Classical Verlet Algorithm

The classical Verlet algorithm can be derived by Taylor expansions of the function to be calculated, expanding forward ($\vec{r}_i(t+\Delta t)$) and backward ($\vec{r}_i(t-\Delta t)$).

$$\vec{r}_i(t+\Delta t) = \vec{r}_i(t) + \Delta t \cdot \dot{\vec{r}}_i + \frac{(\Delta t)^2}{2}\ddot{\vec{r}}_i + \frac{(\Delta t)^3}{6}\frac{\mathrm{d}^3\vec{r}_i}{\mathrm{d}t^3} + \mathcal{O}(\Delta t^4)\,, \tag{4}$$

$$\vec{r}_i(t-\Delta t) = \vec{r}_i(t) - \Delta t \cdot \dot{\vec{r}}_i + \frac{(\Delta t)^2}{2}\ddot{\vec{r}}_i - \frac{(\Delta t)^3}{6}\frac{\mathrm{d}^3\vec{r}_i}{\mathrm{d}t^3} + \mathcal{O}(\Delta t^4)\,. \tag{5}$$

Adding both equations, terms of uneven power of Δt cancel out because of their different signs. So only second and higher order terms have to be considered and the error is only beginning in the forth order term. By the substitution $\ddot{\vec{r}}_i = \frac{\vec{f}_i}{m_i}$, the following equation can be obtained for the next time step:

$$\vec{r}_i(t+\Delta t) = 2\vec{r}_i(t) - \vec{r}_i(t-\Delta t) + \frac{\vec{f}_i}{m_i}\Delta t^2 + \mathcal{O}(\Delta t^4)\,. \tag{6}$$

One can see that the velocity is not needed for calculating the next step but the position of the previous step is used. Often velocity has to be calculated anyways because it is needed for the calculation of macroscopic quantities such as temperature. It can be calculated using the last and the current position using the difference of equations 4 and 5 and then solving for the velocity:

$$v(t) = \frac{\vec{r}_i(t+\Delta t) - \vec{r}_i(t-\Delta t)}{2\Delta t} + \mathcal{O}(\Delta t^3)\,. \tag{7}$$

Here all terms of even order cancel out so the error is of third order. The advantage of the Verlet algorithm is the conservation of the phase space volume (symplecticity) and the reversibility in time. It can be reversed by just replacing $\vec{r}_i(t+\Delta t)$ by $\vec{r}_i(t-\Delta t)$. [1]

The Leapfrog Algorithm

One variation of the classical Verlet algorithm is the leapfrog algorithm. It uses the velocity at the half of a time step for calculating the next step. The algorithm is the following[1]:

$$\vec{r}(t+\Delta t) = \vec{r}(t) + \Delta t \cdot \vec{v}(t+\frac{1}{2}\Delta t) \tag{8}$$

$$\vec{v}(t+\frac{1}{2}\Delta t) = \vec{v}(t-\frac{1}{2}\Delta t) + \Delta t \cdot \vec{a}(t) \tag{9}$$

The new velocity must be calculated first because it is used in the formula for the next step. The current velocity is then simply calculated as the mean

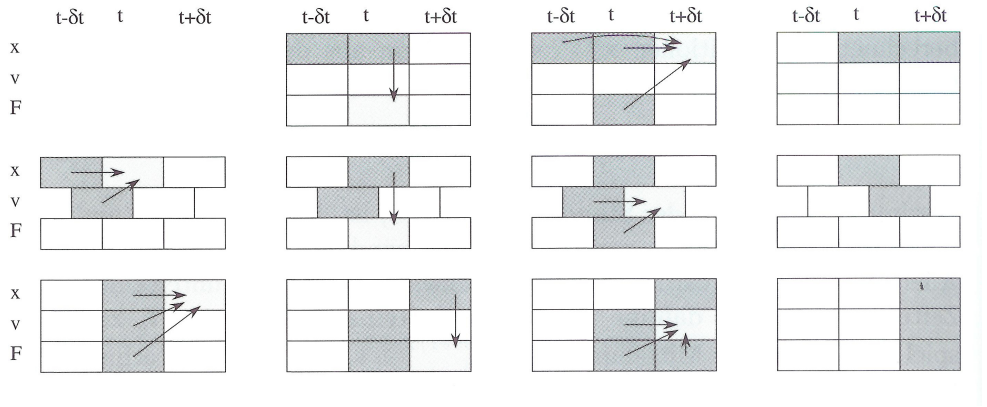

Figure 1: The three variations of the Verlet algorithm: classic (top), Leapfrog algorithm (middle), Velocity Verlet algorithm (bottom) [1]

value of the velocities of the half time step before and the half time step after:

$$\vec{v}(t) = \frac{1}{2}\left(\vec{v}(t + \frac{1}{2}\Delta t) + \vec{v}(t - \frac{1}{2}\Delta t)\right) \tag{10}$$

This is the algorithm used by GROMACS in this tutorial.

The Velocity Verlet Algorithm

Another variation of the Verlet algorithm is the Velocity Verlet Algorithm. It calculates $\vec{r}$, $\vec{v}$ and $\vec{a}$ for the same time. Position and velocity are calculated like this:

$$\vec{r}(t + \Delta t) = \vec{r}(t) + \Delta t \cdot \vec{v}(t) + \frac{1}{2}\Delta t^2 \cdot \vec{a}(t) \tag{11}$$

$$\vec{v}(t + \Delta t) = \vec{v}(t) + \frac{1}{2}\Delta t(\vec{a}(t) + \vec{a}(t + \Delta t)) \tag{12}$$

The accelerations $\vec{a}$ are calculated from the forces $\vec{a} = \vec{f}/m$. To calculate new positions and new velocities at a time $t + \Delta t$, we perform the following steps.

1. Calculate the position after one time step:
   $\vec{r}(t + \Delta t) = \vec{r}(t) + \Delta t \cdot \vec{v}(t) + \frac{1}{2}\Delta t^2 \cdot \vec{a}(t)$

2. Calculate the velocity after half the time step:
   $\vec{v}(t + \frac{1}{2}\Delta t) = \vec{v}(t) + \frac{1}{2}\Delta t \cdot \vec{a}(t)$

3. Calculate the forces and accelerations at time $t + \Delta t$ according to the respective force function.

4. Calculate the velocity after one time step:
   $\vec{v}(t + \Delta t) = \vec{v}(t + \frac{1}{2}\Delta t) + \frac{1}{2}\Delta t \cdot \vec{a}(t + \Delta t)$

We repeat these steps to obtain a trajectory $\vec{r}(t_i)$, i.e., the coordinates at times t_i, of our system of interest.

This algorithm requires the storage of another variable, but the calculation of the velocity (at the same times as the forces and coordinates) in the other two variants requires this as well [1].

2.3 Boundary Conditions

The number of particles in a simulation is limited by computation time and computer memory. For such a system of finite size, we have to define boundary conditions. For small systems, the boundary conditions can have a large effect on the system properties. To avoid such finite-size effects, we aim to simulate systems as part of an infinitely large system.

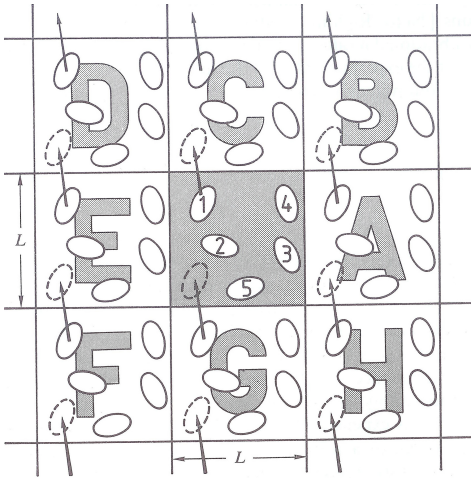

Figure 2: Schematic representation of periodic boundary conditions [1].

To do so, we use periodic boundary conditions. A particle that leaves the simulation box on one side, enters the box again at the opposite side with the same velocity. These boundary conditions are equivalent to simulating a lattice of an infinite number of simulation boxes (see fig. 2).

For short range interactions, we apply the minimum image convention. For each particle, one only consider interactions with particles inside a region,

which is as big as the simulation box but shifted such that the particle is in its center.

2.4 Cut-Off Radius and Neighbor Lists

In order to save computation resources, short-range interactions can generally be neglected for large distances. To do so, a so called cut-off radius r_{cut} is introduced, at which the interaction potential is truncated. Interactions of particles that are further apart than the cut-off radius do not have to be considered in each calculation step. Consequently, we can use neighbor lists, which contain for each particle all neighbors closer than a distance r_{cut}. The runtime of the simulation will now be $\propto N$ instead of $\propto N^2$, but neighbor lists have to be updated repeatedly.

2.5 Thermodynamic Ensembles

Integrating Newton's equations of motion with a certain number of particles in a constant simulation box conserves the total energy and therefore corresponds to a microcanonical ensemble (NVE ensemble). Often, it is necessary to simulate the canonical ensemble (NVT) or the isothermal-isobaric (NPT) ensembles. Several methods have been developed to keep temperature and/or pressure constant using thermostats or barostats, respectively.

Berendsen Thermostat

A simple method to keep the average kinetic energy per particle constant would be to simply rescale the velocities after a certain number of steps by a factor that is determined such that the new kinetic energies sum up to the desired value. This would lead to large jumps in the energy of the system. So the Berendsen algorithm suppresses deviations from the desired temperature T_{ref} not instantaneously but by damping them exponentially with a certain time constant τ, such that

$$\frac{\mathrm{d}T}{\mathrm{d}t} = \frac{T_{\text{ref}} - T}{\tau} \,. \tag{13}$$

Keeping the kinetic energy constant does not necessarily result in constant temperature. The Maxwell-Boltzmann distribution of a proper canonical ensemble is not fulfilled after rescaling. This method can only be used when physical accuracy is not needed, i.e., during equilibration [3].

Nosé-Hoover Thermostat

Nosé's approach to do deterministic molecular dynamics simulations with conserved temperature was to use a modified Lagrangian. He added an additional coordinate s and an "effective" mass Q to the classical N-body Lagrangian given by

$$\mathcal{L}_{\text{Nosé}} = \sum_{i=1}^{N} \frac{m_i}{2} s^2 \dot{\vec{r}}_i^2 - \frac{1}{2} \sum_{i,j,i\neq j} V(\vec{r}_{ij}) + \frac{Q}{2}\dot{s}^2 - \frac{3N}{\beta} \ln s \tag{14}$$

Using this Hamiltonian, we sample an ensemble of constant temperature that also conserves the canonical distribution. At the same time, the trajectories of the particles are deterministic and follow the corresponding equations of motion. These equations have later been simplified by Hoover.

However, the Nosé-Hoover thermostat only works correctly if there is only one conserved quantity in the system and if there are no external forces. By fixing the center of mass, possible external forces can be compensated. Given more conserved quantities the Nosé-Hoover thermostat has to be coupled to a chain of thermostats, the so called Nosé-Hoover chains [3].

Berendsen Barostat

The Berendsen barostat works in the same way as the Berendsen thermostat does. It suppresses deviations from the desired pressure P_{ref} exponentially with a certain time constant τ_P, such that

$$\frac{\mathrm{d}P}{\mathrm{d}t} = \frac{P_{\text{ref}} - P}{\tau_P}\,. \tag{15}$$

This process is implemented by rescaling the size of the simulation box (and all coordinates accordingly) to achieve the desired pressure. Especially for small system, the same problem of an improper canonical ensemble emerges as the pressure does not follow a Maxwell distribution, too [3].

Parrinello-Rahman Barostat

Similar to the Nosé-Hoover thermostat, the Parrinello-Rahman barostat introduces an additional variable to the Hamiltonian, which serves to keep the pressure constant. The Parrinello-Rahman barostat extends this further by making each unit vector of the unit-cell independent so that both the volume and the shape of the box can change. The additional terms are therefore similar to the ones in the Nosé-Hoover thermostat, but more complex.

3 Force Fields

As mentioned in section 2.1 the force fields in a molecular dynamics simulation are interaction potentials from which the forces are calculated via

$$\vec{f}_i = \sum_{i \neq j} \vec{f}_{ij} = -\sum_{i \neq j} \frac{\partial V(\vec{r}_{ij})}{\partial \vec{r}_i} \,. \tag{16}$$

In all-atom force fields, each atom is represented as a mass point moving in space. In so called "united-atom" force fields, hydrogen atoms that are not important for the effects to be focused on are incorporated into the heavier atom they are bound to. One can also go one step ahead and merge several atoms to one simulation bead or "effective atom". This method is called coarse-graining (CG). This course uses such coarse-grained force fields of the MARTINI type as will be described in section 3.3.

3.1 Bonded Interactions

Atoms that are bonded in a molecule are modeled by a special type of interaction. During the course of a simulation, bonded particles remain bonded. Consequently, chemical reactions cannot be modeled using such potentials. Here we describe the most common features of (bio-)molecular force fields:

Covalent bonds: described by a harmonic potential

$$V_b(\vec{r}_i, \vec{r}_j) = \frac{1}{2} k_b (r_{ij} - b)^2 \qquad \text{with} \qquad r_{ij} = \|\vec{r}_i - \vec{r}_j\| \tag{17}$$

or by the so-called Morse potential

$$V_{\text{Morse}}(\vec{r}_i, \vec{r}_j) = D_{ij} \left[1 - \exp\left(-\beta_{ij}(r_{ij} - b)\right)\right]^2 \,. \tag{18}$$

The distance parameter b and the force parameter k_b differ for each interaction type.

Covalent bond angles: in a similar manner described by a harmonic angular potential

$$V_a(\vec{r}_i, \vec{r}_j, \vec{r}_k) = \frac{1}{2} k_\theta (\theta - \theta_0)^2 \tag{19}$$

$$\text{with} \qquad \theta = \arccos \frac{\vec{r}_{ij} \cdot \vec{r}_{kj}}{r_{ij} r_{kj}} \tag{20}$$

or by a simpler potential, which is harmonic in the cosine of the bond angle

$$V_a(\vec{r}_i, \vec{r}_j, \vec{r}_k) = \frac{1}{2} k'_\theta (\cos\theta - \cos\theta_0)^2 \tag{21}$$

Dihedral angles: defined as the angles between the normals $\vec{m}$ and $\vec{n}$ of two planes i, j, k and j, k, l that share two edges (j and k).

$$\phi = \arccos \frac{\vec{n} \cdot \vec{m}}{n\, m} \tag{22}$$

$$\text{with} \quad \vec{n} = \vec{r}_{ij} \times \vec{r}_{kj} \quad \text{and} \quad \vec{m} = \vec{r}_{jk} \times \vec{r}_{ik} \tag{23}$$

The dihedral potential is then described either by a periodic function

$$V_d(\phi) = k_\phi(1 + \cos(n\phi - \phi_0)) \tag{24}$$

for which all minima are equal and diferences must be introduced by an extra interaction between atoms i and l, or by a power series of cosine functions which is then called Ryckaert-Bellemans potential

$$V_{\mathrm{RB}}(\phi) = \sum_{n=0}^{5} C_n \cos^n \phi \tag{25}$$

Improper dihedrals: defined as a harmonic restraint that keeps four atoms in one plane:

$$V_{\mathrm{improper}}(\xi) = \frac{1}{2} k_\xi (\xi - \xi_0)^2 \,. \tag{26}$$

Constraints: Some bonded interactions (mostly hydrogen bonds) are stiff enough to produce only high frequency oscillations ($\nu \ll \frac{k_{\mathrm{B}}T}{h}$). They can be replaced by constraints. These constraints can be realized either by resetting the coordinates after performing a conventional unconstrained step (SHAKE algorithm) or by rewriting the equations of motion in a matrix notation using Lagrange multipliers, so they contain the constraints. Nowadays, the most efficient algorithm of these methods is the LINCS algorithm. It is faster, more robust, more accurate and easier to parallelize than SHAKE, but more complex. The SETTLE algorithm solves the constrained equations analytically and can be used in special applications, for example in water models [2].

3.2 Non-Bonded Interactions

Those atoms that are not bonded in a molecule are assumed to interact only by pairwise additive terms. Mostly they are modeled with Lennard-Jones and Coulomb potentials.

Lennard-Jones interactions: This widely used potential includes the van der Waals interaction, which is attractive and proportional to $-r^{-6}$, as well as a repulsive term representing the Pauli exclusion principle, which is arbitrarily chosen as $\propto r^{-12}$. The energy is usually written in one of the following forms:

$$v_{\mathrm{LJ}}(r) = \frac{C_{12}}{r^{12}} - \frac{C_6}{r^6} = 4\varepsilon \left(\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6} \right) \tag{27}$$

in which σ is the zero point of the function and ε is the depth of the minimum.

A more realistic but also more expensive alternative is the so called Buckingham potential which uses an exponential repulsion instead of the r^{-12} term:

$$v_{\mathrm{rep}} = A \, \exp\left(-B\, r\right) \, . \tag{28}$$

Coulomb interactions: Electrostatic interactions are calculated between the atom centers that are given so called effective or partial charges q_i:

$$V_C(r) = f_{\mathrm{el}} \cdot \frac{q_i q_j}{\varepsilon_r r} \tag{29}$$

Finding suitable partial charges is highly non-trivial. Often they are derived from empirical dipole and quadrupole moments of small molecules. Charges can also be obtained from an analysis of orbital occupations. For force fields, it is best to use potential-derived charges that reproduce the electric potential around the molecule (for example RESP charges).

Coulomb interactions are long-range and one cannot apply a cut-off. In principle, one has to calculate all Coulomb interactions between particles in all periodic images of the simulation box. To that end, methods like Ewald summation and Particle Mesh Ewald method (PME) have been developed. However, in this course we use a coarse-grained force field (MARTINI), where Coulomb interactions are not calculated explicitly but treated as effective short-range potentials. Therefore, we do not discuss these methods here although they are of great importance for atomistic simulations.

Exclusions: Non-bonded interactions are not applied to pairs of atoms that already form a covalent bond. Often interactions with those atoms are excluded when connected indirectly via two or more covalent bonds. Sometimes the latter ones are included but in a modified form, depending on the exact realization of the force field [2].

3.3 Coarse-Graining with MARTINI

Simulations considering each single atom are very expensive and in order to reach longer scales in space and time, coarse-graining (CG) can be used to decrease computational effort. Coarse-graining means merging several atoms to one simulation bead or "effective atom" so the number of particles to be simulated decreases. There are several CG approaches that differ widely in the accuracy of their mapping i.e. how many "real" atoms are mapped to one bead. A CG force field is usually parameterized by matching it to all-atom results. This is not trivial, because by merging the atoms, the potential energy landscape becomes smoother, which changes the dynamics of the simulation drastically. The conversion of the speeds of several effects on different scales is often different, so there is no unique relation between the timescales of all-atom simulations and coarse-grained simulations.

The coarse-grained model used in this work is the MARTINI model introduced by Marrink et. al [5]. It was originally designed for lipids and has been extended to biochemical compounds in general. The most recent version of the model, MARTINI 3, has been recently released and will be used in this course [6].

The mapping of the MARTINI model is basically four-to-one, which means that four heavy atoms are mapped to one interaction site. For ring-like structures, a finer mapping of two or three atoms to one site is chosen in order to represent their geometry more accurately. Liquid water fits into this scheme due to the tetrahedral coordination and four water molecules are represented by one CG bead. Hard ions such as Na+ and Cl- are modeled as tiny beads and thus lack an hydration shell.

The MARTINI model comprises four main types of particles (abbreviated by capital letters): polar (P), non-polar (N), apolar (C), and charged (Q). Additionally, numbers indicate the degree of polarity from 1 (low polarity) to 5 (high polarity). In the newest version, special bead types were included, including halo compounds (X), water (W) and divalent charged molecules (D). Additional properties of the beads, such as their hydrogen bonding capability, can be specified by using labels. These are denoted by lowercase letters, for instance "d" for hydrogen donor and "a" for acceptor. A total of 29 different "building blocks" are available.

The non-bonded beads interact by a Lennard-Jones 12-6 potential. For the interaction parameters, see the MARTINI 3 publication [6]. The charged groups of type Q and D are additionally assigned a charge $\pm e$. Their Coulomb interaction is screened by using a dielectric constant $\varepsilon_{\text{rel}} = 15$. All non-bonded interactions are cut-off at a distance $r_{\text{cut}} = 1.1\,\text{nm}$ and shifted from $r_{\text{shift}} = 0.9\,\text{nm}$ to r_{cut}.

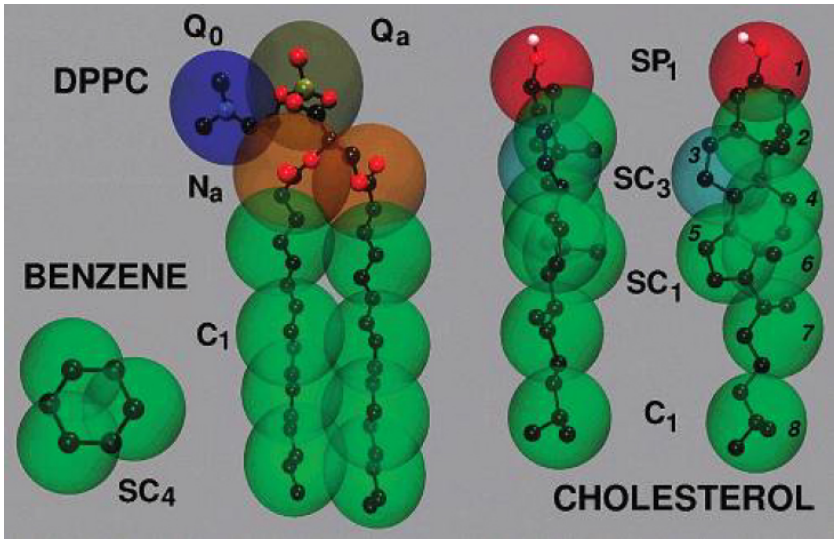

Figure 3: Examples for MARTINI mapping from the chemical structure to the coarse-grained model. The coarse grained bead types, which determine their relative hydrophilicity, are indicated. The prefix S denotes a special class of coarse-grained sites introduced to model rings. [5]

In the MARTINI model the neighbor list searching radius r_{list} is equal to the cut-off radius. This leads to a systematic error as atoms are taken into account, which have moved out of the respective area between two neighbor list updates and some are not taken into account that have moved in. Therefore the neighbor list algorithm (in the form as implemented in GROMACS) with updates every 10 steps is considered to be a part of the MARTINI model.

The bonded interactions are modeled by harmonic bond potential, angle potentials, proper dihedrals, and improper dihedrals.

4 Self-Assembly of a Lipid Bilayer

The goal of this course is to learn how to setup, run, and analyze molecular dynamics simulations.

In the following, we will simulate and study a lipid bilayer, the fundamental component of biological membranes. We will use the MARTINI force field, which is a coarse-grained force field specifically developed for the simulation of lipids, and more recently also extended for the simulation of proteins embedded in the lipid bilayer. We use the GROMACS software package, which is one of the most popular molecular dynamics engines available for free.

Make sure that you keep all information you need for writing the report afterwards. You can find our expectations and hints on writing a good report in the appendix.

4.1 Overview of GROMACS File Structure

To setup and run a GROMACS simulation, we have to prepare the configuration of the system we want to simulate, decide what type of simulation we want to perform, and specify the parameters. All files needed at this stage are text files, and can be modified with any text editor, e.g. vim or emacs. Doing so we obtain a binary file with extension "**.tpr**". This file is not human-readable anymore, but contains all information necessary to actually run the simulation.

In a simulation setup, we need a parameter file, a structure file, and a topology file. These different file types are explained below. While the format and names of these files vary between different molecular dynamics programs, the basic concept is essentially the same.

1. **Parameter file (.mdp)** – Input file containing all parameters of the simulation. A molecular dynamics simulation is a technically very complex object, and one has to specify many parameters. Open the minimization.mdp file and try to give a look to the different parameters (the file is heavily commented). It is not necessary and indeed impossible to understand the meaning of each of those parameters at the beginning, and only time and experience will bring some familiarity with most of them. Some of them, though, are of primary importance.

   `integrator` – The numerical algorithm used to evolve the system.

   `dt` – The time step to be used in the propagation of the system.

   `nsteps` – Total number of time steps to be used, which determines the total duration of the simulation.

It is important to note and keep in mind that the MARTINI force field was developed and optimized to be used with a given set of parameters. For this reason, it is wise and safe not to change the parameters that are recommended by the authors of the force field, unless one is really sure of the modification they are going to apply.

2. **Structure file (.gro)** – The structure file contains the initial configuration of the system.

3. **Topology file (.top)** – In this file one has to specify which force field parameters have to be used and the type and number of molecules of different kind that are present in the .gro file. Depending on the system, the topology file can be created by using some scripts or has to be build up by hand. Note that the force field parameters are used by explicitly including the .itp files `martini_v3.0.itp` and `martini_v3.0_lipids.itp`. These are again text files that contain all the parameters of a given force field with a well defined format. The "**.itp**" files have to be contained either in a system folder of GROMACS, or in the current working folder.

These files (.mdp, .gro, .top) contain all information that is needed to run a simulation.

We will perform some simple simulations and learn step by step the key ingredients of a molecular dynamics simulation of a biomolecular system.

4.2 Preliminaries

For the present tutorial, some basic familiarity with Linux operating systems and the Bash shell is required. If you do not have any experience with any of these topics, please consult the appendix for brief introductions.

You should have a folder containing the following files needed to set up simulations:

- `water.gro`
- `dspc.top`
- `dspc_single.gro`
- `minimization.mdp`
- `martini3.mdp`
- `martini_v3.0.0_phospholipids_v1.itp`

- `martini_v3.0.0_solvents_v1.itp`
- `martini_v3.0.0.itp`

for analysis and visualization,

- `axes.py` - Show coordinate axes in PyMOL (run `axes.py`).
- `cg_bonds-v5.tcl` - Tcl script to show MARTINI bonds in VMD.
- `README_cg_bonds.txt` - Explaining the usage of `cg_bonds-v5.tcl`.
- `do-order-multi.py` - Python script to calculate the P_2 order parameter (see Eq. (30))

and the course description, the original MARTINI paper, and the GROMACS 2016 manual

- `Versuch13_Molecular_Dynamics.pdf`
- `2007JPhysChemBMarrink.pdf`
- `gromacs-manual-2022.pdf`

For your convenience, please add the GROMACS path and VMD path to your search path, i.e., execute the following command in the shell (terminal). The "$" sign indicates the so-called command prompt, and must not be typed.

```
$ export PATH="/home/net/bin/:$PATH"
```

Whenever you open another shell (terminal), this command has to be executed again[2].

4.3 System Setup

To simulate the self-assembly of a lipid bilayer, we need a starting configuration of a mixture of lipids and water. Structures of a lipid molecule and a water molecule are provided by MARTINI. We put a number of lipid molecules randomly in a simulation box and fill up the remaining void space with water.

The structure of a single DSPC lipid is given in the file `dspc_single.gro`. Files with the extension "**.gro**" are text files of fixed format and contain all

[2]Bash experts can add those lines to their `.bashrc` file so it will be automatically executed when opening a new shell.

information necessary to determine the structure of a molecule. Take a look at the file (open it in a text-editor) and try to understand the content.

Now let us visualize the lipid, to have an idea of what it looks like. In order to visualize molecules, we will use the program "Visual Molecular Dynamics" (see appendix) or VMD for short, e.g.,

```
$ vmd dspc_single.gro
```

To exit VMD type "quit", or choose "File" and "Quit" form the drop down menu. To set a different representation, go to "Graphics" and "Representations..." in the drop down menu and select, for example, "VDW" as drawing method.

We use this structure to build our simulation box, which contains all elements of the system we want to simulate. For simulations with GROMACS, we need a structure file (.gro) of the simulation box. We do not have to create such a file by hand, which would be tedious and prone to error. Indeed, most of these mundane but frequent tasks can be carried out by using scripts and programs provided by GROMACS or other sources. These programs are executed in a shell and usually have multiple command-line parameters.

To set up the simulation box, we use the program `gmx insert-molecules`. To read the help text of this program, execute `gmx insert-molecules -h` in a shell. Now run the following line to generate a box containing 128 lipid molecules in random positions.

```
$ gmx insert-molecules -ci dspc_single.gro -nmol 128 -box 7.5
   7.5 7.5 -try 500 -o 128_noW.gro
```

Filling the simulation box randomly with molecules will lead to high energy configurations, i.e., molecules that are very close to each other. In order to relax such high energy configurations, we use an optimization algorithm (e.g., steepest descent) to minimize the energy of the system. Performing an energy minimization follows the same steps as performing a molecular dynamics simulation. The difference between these two modes of operation is that we either have to specify a minimization algorithm or an integrator.

The program `gmx grompp` will combine these files in a single "**.tpr**" file, i.e.,

```
$ gmx grompp -f minimization.mdp -c 128_noW.gro -p
   dspc-noW.top -o dspc-min-init.tpr
```

The molecular dynamics engine of GROMACS, the program that is actually performing the simulation, is `gmx mdrun`. To run the energy minimization, type

```
$ gmx mdrun -deffnm dspc-min-init -v -c 128_minimised.gro
```

The minimization should be very quick and should remove the most severe clashes between lipid molecules.

So far our system contains only lipids, but the self-assembly of a bilayer is an entropically driven phenomenon that is possible only in the presence of water. We will use the `gmx solvate` tool to add MARTINI water, which is a coarse-grained water model where one bead corresponds to four atomistic water molecules:

```
$ gmx solvate -cp 128_minimised.gro -cs water.gro -o
   waterbox.gro -maxsol 1000 -radius 0.21
```

Once we have added water to the box, we have to minimize the potential energy of the structure again. Before using `gmx grompp`, open the topology file in an editor and modify it correspondingly to reflect the presence of the water molecules (hint: ";" indicates a comment). Again, we minimize the energy of the structure using

```
$ gmx grompp -f minimization.mdp -c waterbox.gro -p dspc.top
   -o dspc-min-solvent.tpr
$ gmx mdrun -deffnm dspc-min-solvent -v -c minimised.gro
```

Q 1: Setting up the system

(a) How are the lipids placed in the box? (Hint: see option `-try`)

(b) In contrast to that, how are the water molecules put into the box? (Hint: see option `-cs`)

(c) Why do we have to use the `-radius` flag? What happens if we do not use it?

4.4 Running the Molecular Dynamics Simulation

We are ready to run a molecular dynamics simulation of the system in the canonical ensemble starting from the energy minimized configuration of lipids and water. We will perform two simulations. First we will equilibrate the system and then we will perform a data-production run of the formed bilayer.

4.4.1 Equilibration Run

Such a simulation is different compared to a simple energy minimization, and consequently the input file has to change. Have a look at the `martini3.mdp`

file to see the differences and find out how long you will simulate the system. To prepare the new **.tpr file** type

```
$ gmx grompp -f martini3.mdp -p dspc.top -c minimised.gro -o
  dspc-eq.tpr
```

Now we can run the simulation by typing

```
$ gmx mdrun -deffnm dspc-eq -v
```

This step can be quite lengthy, and depending on the amount of available cores it might take several minutes (GROMACS should provide an estimate of the remaining time). To make sure that the simulation behaves as expected, you can use VMD to check the trajectory visually while it is still running, e.g., in a separate terminal (with proper search path set, see above) type

```
$ vmd minimised.gro dspc-eq.xtc
```

To not get confused by periodic boundary conditions, VMD can display periodic images (see appendix).

In this first run a bilayer should form. However, it might not be formed perfectly. In this case, extend the simulation by 9000 ps:

```
$ gmx convert-tpr -s dspc-eq.tpr -extend 9000 -o dspc-eq.tpr
$ gmx mdrun -v -cpi dspc-eq.cpt -deffnm dspc-eq
```

If necessary, repeat this until you have a nice and stable bilayer.

Q 2: Simulation parameters

(a) For what time interval have you simulated the system?

(b) What are the two most important differences compared to the previous `.mdp` parameter file?

4.4.2 Production Run

We do not want our analyisis be influenced by the equilibration phase, so we use the end point of our first simulation, **the equilibration run**, for a new simulation, i.e., **the production run**. The resulting trajectory will be used to analyse the properties of the bilayer. The choice of the simulation time depends on the observable of interest, the available resources, and the desired sampling. It should be long enough to obtain meaningful averages of those properties. You may adapt it via the number of steps in the "**.mdp**" file. Run the simulation by entering the following commands:

```
$ gmx grompp -f martini3.mdp -p dspc.top -c dspc-eq.gro -o
   dspc-pr.tpr
$ gmx mdrun -v -deffnm dspc-pr
```

where `dspc-eq.gro` has been generated by your first simulation and contains the state of your system at the last frame. If during analysis you get the impression that the simulation was not long enough, you can extend it as you did with the equilibration run.

Q 3: Observables

(a) How will you see during analysis that an observable is equilibrated?

(b) How long should a simulation be to get meaningful averages?

4.5 Analysis

4.5.1 Visual Analysis

While performing a simulation, one should check the trajectories visually, for instance using VMD. We want to make sure that the evolution of the system looks reasonable and we check for pathological behavior, which would indicate technical problems or mistakes in the input files. For the equilibration run, check whether the bilayer has formed, how much time that took and whether it is stable. Find out the orientation of the bilayer and note the direction normal to the bilayer plane. You will need it later.

Q 4: Jumping molecules

(a) Some molecules are jumping across the boundaries of the box. What is the reason for that?

4.5.2 Energy as a Function of Time

The GROMACS tool `gmx energy` can read and extract many parameters of the simulation from the binary "**.edr**" file. Using for example

```
$ gmx energy -f dspc-eq.edr -o total_energy_eq.xvg
```

we can choose, which quantities should be extracted. `gmx energy` produces an xmgrace file with the extension **.xvg**, which can viewed using

```
$ xmgrace total_energy_eq.xvg
```

or `gnuplot` (see appendix). Write out the total energy as a function of time (Option 7 Total-Energy) for both the equilibration and production run (to separate files) and plot them. Describe and explain the correlations of the total energy with the formation of the lipid bilayer.

4.5.3 Order Parameter for Self-Assembly of the Bilayer

We now want to calculate a quantity that measures the progression of the system from a disordered phase, where lipids and water are randomly mixed, to an ordered phase, where the bilayer has formed (equilibration run!). Such a quantity is usually known as an order parameter. How can we follow the formation of the bilayer? What could we measure to monitor the increasing order of the bilayer in time? One possibility is to use the so called P_2 second-rank order parameter, that measures the alignment with a specified direction. It is defined as

$$P_2 = \frac{1}{2}\left(3\langle\cos^2\theta\rangle - 1\right) , \tag{30}$$

where θ is the angle formed by a given bond stretching between two atoms and the normal to the plane of the bilayer. Angle brackets indicate the average over all lipids for a single frame.

Let us visualize again the structure of a single lipid molecule using VMD, by typing

```
$ vmd dspc_single.gro
```

Check the naming of the beads and how they are connected so you can understand the output of the following analysis.

In order to calculate the value of P_2 on all bonds along the simulated trajectory, we will use the `do-order-multi.py` script. For this purpose, create an analysis folder, and copy there the equilibration trajectory as `traj.xtc` and the "**.tpr**" file of the molecular dynamics simulation as `topol.tpr` (the script accepts only the files with these names) and the Python script itself. The command to run the analysis is

```
$ python2 do-order-multi.py traj.xtc 0 150000 10 0 0 1 128
   DSPC
```

with the following arguments (in this order):

- the trajectory file,
- the initial time in the trajectory to start the analysis (in ps),
- the time where to stop the analysis (in ps),
- the number of frames to skip along the trajectory,
- the axis normal to the bilayer
  ("1 0 0" for X, "0 1 0" for Y, and "0 0 1" for Z),
- the number of lipid molecules,
- the type of lipid molecules.

The script produces the file `order.dat`. The file contains a table with the values of P_2 as function of time for each bond in the specific lipid molecule. Plot the time series of P_2 for some representative bonds you think may be important for our problem and describe what we can learn from them.

Q 5: Order parameter

(a) How does P_2 behave in different situations? What is the value of P_2 for a bond that is parallel/orthogonal to the bilayer normal?

(b) What characterizes a good order parameter?

(c) Which bond gives the best order parameter to describe the formation of the bilayer?

For all following analysis, go back to the directory with your original trajectories, because only the production run should be used.

4.5.4 Area per Lipid

An important parameter of lipid membranes is the area per lipid, quantifying how densely the lipids are packed. Approximately half of the lipids should be located in each leaflet of the bilayer. Consequently, the area per lipid can be estimated using the total number of lipids and the area of the membrane, The latter is obtained from the box dimensions. Use `gmx energy` to obtain the average values of the two box dimensions in which the lipid bilayer has formed and calculate the average area per lipid.

4.5.5 Bilayer Thickness

Another important physical property of the bilayer is the thickness. How can we define it? A possible solution is to take the difference between the average positions of some heavy atoms in the head groups of the lipids, for instance the phosphate groups represented by beads with names starting with "P". We must therefore create an index file containing the atom numbers corresponding to all phosphate atoms. This can be done by using the `gmx make_ndx` tool. Execute

```
$ gmx make_ndx -f dspc-pr.gro -o index_P.ndx
```

and select only the P beads in the configuration by choosing "atom" and all phosphate beads by typing

```
$ a P*
```

The tool should find 128 atom types of this kind. Type `q` to save and exit the program. To make sure that the entire bilayer is inside of the box and not broken over the periodic boundary conditions, we will run a series of commands to center the membrane in the box. For this, we will first select one bead in the tail of the first lipid (bead 14), and center it in the box. Then, some corrections need to be made to place the center of mass of all molecules in the box.

```
$ echo -e 'a14 \n q' | gmx make_ndx -f dspc-pr.gro -o
    index.ndx
$ echo -e 'a14 \n System \n q' | gmx trjconv -f dspc-pr.xtc
    -center -o dspc-pbc01.xtc -s dspc-pr.tpr -n index.ndx
$ echo -e 'System' | gmx trjconv -f dspc-pbc01.xtc -pbc mol
    -o dspc-pbc02.xtc -s dspc-pr.tpr
$ echo -e 'DSPC \n System \n q' | gmx trjconv -f
    dspc-pbc02.xtc -center -o dspc-pbc03.xtc -s dspc-pr.tpr -n
    index.ndx
$ echo -e 'System' | gmx trjconv -f dspc-pbc03.xtc -pbc mol
    -o dspc-pbc04.xtc -s dspc-pr.tpr
```

Now we will invoke `gmx density` to calculate the density of the P beads along the axis normal to the bilayer. In the following we assume that the membrane normal is in direction of the z-axis. Before using this tool check carefully the orientation of the membrane normal and adapt the commands below accordingly. To calculate the density along the z-axis, we use the option `-d Z` and execute

```
$ gmx density -f dspc-pbc04.xtc -n index_P.ndx -s dspc-pr.tpr
    -d Z -center -o density-P.xvg
```

The `-center` option will allow you to center the density around the center of mass of a given group. The "P*" group should appear among the possible choices, select it and let the program calculate the desired density. This command will produce an `.xvg` file, which can be plotted for instance with `gnuplot` (see appendix), by typing (you can leave gnuplot afterwards by entering `quit`):

```
$ gnuplot
$ p 'density-P.xvg' u 1:2 w l
```

Two peaks should be clearly visible, and the bilayer thickness can be estimated from their locations.

Calculate also the density along the membrane normal for water ("W") and carbon beads ("C*"). Plot all three densities in the same figure and describe the result.

Q 6: Density

(a) Which kind of density did we plot? What are the units of measure?

(b) Do the densities overlap? What does this mean?

4.5.6 Lateral Diffusion of Lipids

The bilayer is a sort of liquid and the lipids are free to diffuse within each leaflet. The mean squared displacement (MSD) is given by

$$\mathrm{MSD}(\tau) = \left\langle [\vec{r}\,(\tau) - \vec{r}(0)]^2 \right\rangle , \tag{31}$$

where $\vec{r}$ is the position vector of a lipid. According to Einstein's classical result, $\mathrm{MSD}(\tau)$ is proportional to the diffusion constant, i.e.,

$$\mathrm{MSD}(\tau) = 4D\tau, \tag{32}$$

where the factor 4 accounts for the diffusion in two dimensions. The diffusion constant can be determined from the slope of a linear fit to $\mathrm{MSD}(\tau)$.

For the trajectory from the production run, remove the periodic boundary conditions by "unwrapping" the motion of each lipid molecule:

```
$ gmx trjconv -f dspc-pr.xtc -s dspc-pr.tpr -pbc nojump -o
   traj_nj.xtc
```

Calculate the lateral MSD using

```
$ gmx msd -f traj_nj.xtc -s dspc-pr.tpr -lateral z -n
    index.ndx -o msd.xvg
```

where `z` is the membrane normal. This script returns a file `msd.xvg`, and by plotting its content we should find a region in which the MSD is a straight line. Fit Eq. 32 to this linear regime of the plot. Account for the non-linear behavior in the initial regime by including an offset, i.e., fitting to $\mathrm{MSD}(\tau) = 4D\tau + a$ with both D and a as free parameters. The fitting can be done, e.g., with `gnuplot` or with `gmx msd`. The latter will do some supplementary statistics, too.

Q 7: Mean squared displacement

(a) Why do we need to remove the motion of the center of mass of the box?

(b) Is the MSD a straight line over all its domain? What could be a possible reason for any deviation from a straight line?

Appendix A Basic Shell Commands

Various different shells exist, which allow to execute commands from a terminal window. For this course we recommend `bash`, which you can start by typing

```
$ bash
```

in a terminal window. In the following, a short list of the most useful commands is presented.

- `ls` - Show lists of files or information on the files.

`ls file`	Does the file exist?
`ls -l file`	Show information about the file.
`ls *.txt`	Show all files ending in "**.txt**".
`ls dir`	Show contents of directory.

- `cd` - Change directory.

`cd`	Go to home directory.
`cd ~/papers`	Go to /home/your_user_name /papers.
`cd dir`	Go to directory 'dir' (relative).
`cd ..`	Go to the directory above.
`cd -`	Go to the last directory you were in.

- `pwd` - Show current working directory.

Appendix B Plotting with Gnuplot

Plotting with gnuplot is easy and fun! Be sure to have a data file with numbers arranged in columns, for instance for a time series $x(t)$, a first column with the values of t and a second column with the values of x. Type

```
gnuplot
```

to open the program. To plot the curve using columns 1 and 2 of the data file use

```
plot 'filename' using 1:2 with lines
```

Gnuplot was designed for quick plotting, and key words can be contracted unless there is any ambiguity. So the previous command becomes

```
p 'filename' u 1:2 w l
```

If we have the sightliest more complicated situation of having t, $x(t)$ and $y(t)$ as column 1, 2 and 3 in the same file, the plotting command would look like

```
p 'filename' u 1:2 w l t "x(t)", "" u 1:3 w l t "y(t)"
```

where t stands for title, which adds a label to a curve and which increases the readability of plots with many curves. To plot into a file, you have to set the file name

```
set output "filename.ps"
```

the output format,

```
set terminal postscript color enhanced
```

and execute a plot command or replot the previous plot using

```
replot
```

To plot on the screen again, use

```
set output
set terminal x11
```

Gnuplot can also be used for fitting, e.g.,

```
fit [10:100] a*x+b 'data.dat' u 1:2 via a,b
```

fits a linear function to the data in "data.dat" in the range between 10 and 100.

We recommend to follow any of the many excellent tutorials that can be found online in order to learn more advanced features. For example, http://www.mathematik.hu-berlin.de/~ccafm/lehre_BZQ_Numerik/allg/gnuplotkurs.html.

Appendix C Visual Molecular Dynamics

The program Visual Molecular Dynamics or VMD is freely available for various operating systems at
http://www.ks.uiuc.edu/Research/vmd/
For a tutorial see:
http://www.ks.uiuc.edu/Training/Tutorials/vmd/tutorial-html/

Here are some settings that will be particularly useful in this tutorial:

- Depict the atoms as beads:
  Graphics → Representations...
  In `Drawing Method`, select `VDW`.

- Show periodic images:
  Graphics → Representations...
  Go to the tab `Periodic` and tick the dimensions in which to show periodic images.

- Show bonds of coarse-grained molecules:
  This is described in the file `README_cg_bonds.txt`

Additionally it is nice to know the following keyboard shortcuts:

- Press `p` and left-click an atom. This will "**p**ick" the clicked atom and print out its information (name, type, residue, postions, ...) into the terminal.

- Press `r` to switch back to the "**r**otate" mode. This is the default and enables to you rotate the view.

- Press `=` to reset the camera on all visible atoms.

- Press `c` and left-click an atom. This will "**c**enter" the atom, making all rotations relative to it.

Appendix D GROMACS Tools Used in this Tutorial

GROMACS is freely available and can be downloaded from
http://manual.gromacs.org/documentation/
This tutorial assumes you are using version 2016.3.

insert-molecules Generates and modifies a simulation box.

grompp GROMACS preprocessor. Merges text file containing input parameters and initial configuration in a runable binary file.

mdrun GROMACS main MD engine.

energy Extract quantities stored in the "**.edr**" file during the simulation.

make_ndx Makes an index file, i.e. a file with a label associated to a group of atoms.

density Calculates density of a given group along a direction.

trjconv Converts GROMACS trajectories between different formats, changes the number of frames, removes PBC conditions and much more!

msd Calculates MSD and estimates linear and lateral diffusion of a given type of atoms or molecules.

Appendix E How to Write a Good Report

After the course, you are supposed to write a report about what you have done. This report should convince the reader that you properly understand the topic, performed the simulations and the analysis correctly, and that you are able to present your results in an understandable and appealing way. You are free to choose the specific format or text processing software, but you should definitely consider the following guidelines.

E.1 Preparation

Writing a good report starts already before and during the course.

(a) Read the instructions before the course and make sure you understand the general methods and aims of the course. Try to get familiar with the methods and software you will use.

(b) Make sure you keep all data and note everything that might be important for the later analysis and report. Having read all of the instructions helps a lot.

(c) Check your data before you leave. It can be very frustrating to find that your analysis script has produced only empty files.

E.2 Content

You want the reader to follow your explanations, so you should follow the established structure of a scientific report.

(a) **Introduction:** Start with a short introduction about the topic and the main goal of your work (roughly 1/4 page).

(b) **Basics:** Explain the basics of the system under investigation and of the methods you use.

(c) **Results and Discussion:** Describe what you have done in particular and what follows from this. Do not just repeat Bash commands or lines of code from the instruction, but describe your work in a way an expert of the field who is not familiar with the specific software can understand it. Show your results in a traceable way, e.g. give the formulas you used, explain your choices of parameters, and stick to an understandable style (see below).

(d) **Conclusions:** Shortly describe what you have done and what are your main results.

In addition, we pose some questions throughout the instruction (marked with Q and numbered). Answer them in your report referring to them with the same numbering scheme.

E.3 Style

To convey your message and get a good grade, make sure the reader has no problems reading your texts and understanding your plots.

(a) **Language:** You can write your report in German or English (please do not mix). In any case, make sure your language is scientifically adequate but not over-complicated.

(b) **Figures:** Images and plots as well as their labels should not be too small. All lines and labels should be easy to see and distinguish. Do not forget to label the axes of plots with the plotted quantity and to provide a legend. Plots without proper axis labels are useless and will be treated as non-existent. Each figure should also have a short but meaningful caption explaining what is shown. A good figures along with its caption is self-explanatory.

(c) **Units:** This is not a course in pure mathematics. So you have to use (correct!) units when giving results and in your plot labels.

References

[1] Mike P. Allen and Dominik J. Tildesley. *Computer Simulation of Liquids*. Oxford Science Publications. Clarendon Press, Oxford, 1 edition, 1987. 2.1, 2.2, 2.2, 1, 2.2, 2

[2] Herman J.C. Berendsen. *Simulating the Physical World*. Cambridge University Press, 2007. ISBN: 0-521-83527-5. 3.1, 3.2

[3] Daan Frenkel and Berend Smit. *Understanding Molecular Simulation*. Academic Press, San Diego, second edition, 2002. 2.1, 2.2, 2.5, 2.5, 2.5

[4] Siewert J. Marrink. Martini tutorials: lipids. 1

[5] Siewert J. Marrink, H. Jelger Risselada, Serge Yefimov, D. Peter Tieleman, and Alex H. de Vries. The MARTINI Force Field: Coarse Grained Model for Biomolecular Simulations. *Journal of Physical Chemistry B*, 111(27):7812–7824, July 2007. 3.3, 3

[6] Paulo CT Souza, Riccardo Alessandri, Jonathan Barnoud, Sebastian Thallmair, Ignacio Faustino, Fabian Grünewald, Ilias Patmanidis, Haleh Abdizadeh, Bart MH Bruininks, Tsjerk A Wassenaar, et al. Martini 3: a general purpose force field for coarse-grained molecular dynamics. *Nature methods*, 18(4):382–388, 2021. 3.3, 3.3

[7] Martin Vögele. Coarse-grained models for polyelectrolytes. Master's thesis, University of Stuttgart, May 2014. 1